

DAY 11

NAME: Nandhini V

DATE: 19-08-2025

Task:

Add an additional table called users. put api post request to insert user into it in fast api. Additionally, when request to list music api is called, only a valid user is allowed to get the response else raise HTTPException with error code.

Code:

Day10Task.py

```
from sqlalchemy import create_engine
from sqlalchemy.dialects.sqlite import *
# SQLALCHEMY_DATABASE_URL = "sqlite:///./song_db.db"
from dotenv import load_dotenv

import os
load_dotenv()

SQLALCHEMY_DATABASE_URL = os.getenv("Songs_Db_Url")

engine = create_engine(SQLALCHEMY_DATABASE_URL, connect_args =
{"check_same_thread" : False})

from sqlalchemy.orm import sessionmaker, Session, declarative_base
session = sessionmaker(autocommit = False, autoflush = False, bind = engine)

Base = declarative_base()

from sqlalchemy import Column, Integer, String
from pydantic import BaseModel

class songs(Base):
    __tablename__ = 'songs'
    Id = Column(Integer, primary_key=True, index=True)
    Name = Column(String(50), nullable=False)
    Movie = Column(String(30), nullable=False)
    Duration = Column(Integer, nullable=False)
    Base.metadata.create_all(bind=engine)

class users(Base):
    __tablename__ = 'users'
```

```
Id = Column(Integer, primary_key=True, index=True)
Username = Column(String(50), nullable=False)
Password = Column(String(50), nullable=False)
Base.metadata.create_all(bind=engine)
```

```
from fastapi import FastAPI, Request, Depends, HTTPException, status
import uvicorn
from fastapi.templating import Jinja2Templates
app = FastAPI()
```

```
templates = Jinja2Templates(directory="my_web_pages")
```

```
class song(BaseModel):
    Id:int
    Name: str
    Movie:str
    Duration: int
    class Config:
        orm_mode = True
```

```
class user(BaseModel):
    Id:int
    Username: str
    Password: str
    class Config:
        orm_mode = True
```

```
def get_db():
    db = session()
    try:
        yield db
    finally:
        db.close()
```

```
@app.post("/add_user")
def add_user(u: user, db: Session = Depends(get_db)):
    usr = users(Username=u.Username, Password=u.Password)
    db.add(usr)
    db.commit()
    db.refresh(usr)
    return {"msg": "User created", "user": usr.Username}
```

```
async def ValidateExistence(uname:str, db: Session = Depends(get_db)):
    usr = db.query(users).filter_by(Username = uname).first()
    if usr == None:
        raise HTTPException(status_code = 401, detail = "Invalid username or password")
    else:
        return uname
```

```
@app.post('/add_song')
```

```

def add_song(s: song, db: Session = Depends(get_db), validated_user:str =
Depends(ValidateExistence)):
    sg = songs(Name=s.Name, Movie=s.Movie, Duration=s.Duration)
    db.add(sg)
    db.commit()
    db.refresh(sg)
    return sg

@app.get("/songs_list")
def view_songs(request: Request, db: Session = Depends(get_db), validated_user:str =
Depends(ValidateExistence)):
    records = db.query(songs).all()
    return templates.TemplateResponse("songs.html", {"request": request, "songs": records})

@app.get("/getBy/{_id}")
async def getBy(_id:int, db: Session = Depends(get_db), validated_user:str =
Depends(ValidateExistence)):
    sg = db.query(songs).filter_by(Id = _id).first()
    if sg == None:
        raise HTTPException(status_code=404, detail="Song not found")
    return sg

if __name__ == "__main__":
    uvicorn.run("Day10Task:app", host = "127.1.1.0", port = 8000, reload = True)

```

.env

Songs_Db_Url = "sqlite:///./song_db.db"

Output:

Adding an user

Request body required

application/json

Edit Value | Schema

```

{
  "Id": 0,
  "Username": "Riya",
  "Password": "Payoda"
}

```

Execute

200

Response body

```
{
  "msg": "User created",
  "user": "Riya"
}
```

Response headers

```
content-length: 36
content-type: application/json
date: Tue, 19 Aug 2025 12:10:21 GMT
server: uvicorn
```

After adding an user - Database

Table: users

	<u>Id</u>	Username	Password
	Filter	Filter	Filter
1	1	Nandhini	Payoda
2	2	Riya	Payoda

Adding a song using valid username

uname * required
string
(query)

Nandhini

Request body required application/json

Edit Value | Schema

```
{
  "Id": 0,
  "Name": "Vaseegara",
  "Movie": "Minnale",
  "Duration": 400
}
```

Execute

Code	Details
200	Response body <pre>{ "Duration": 400, "Name": "Vaseegara", "Id": 8, "Movie": "Minnale" }</pre> Response headers <pre>content-length: 60 content-type: application/json date: Tue, 19 Aug 2025 12:13:30 GMT server: uvicorn</pre>

Adding a song using valid username - Database

Table: songs

	<u>Id</u>	Name	Movie	Duration
	Filter	Filter	Filter	Filter
1	1	Hosanna	VTV	300
2	2	Kanimaa	Retro	400
3	3	Sawadeeka	Vidaamuyarchi	250
4	4	Mundhinam	Vaaranam Aayiram	300
5	5	Kannai Vittu	Iru Mugan	400
6	6	Jinguchaa	Thug Life	350
7	7	Golden ...	NEEK	350
8	8	Vaseegara	Minnale	400

Trying to add a song with invalid user

Name	Description
uname * required string (query)	Rajesh
Request body required	
Edit Value Schema	
<pre>{ "Id": 0, "Name": "Vaseegara", "Movie": "Minnale", "Duration": 400 }</pre>	

Code	Details
401 <i>Undocumented</i>	Error: Unauthorized Response body <pre>{ "detail": "Invalid username or password" }</pre> Response headers <pre>content-length: 41 content-type: application/json date: Tue, 19 Aug 2025 12:15:30 GMT server: uvicorn</pre>

Listing songs with valid user

Name	Description
uname * required string (query)	Nandhini
<button>Execute</button>	

127.1.1.0:8000/songs_list?uname=Nandhini

Songs List

Id	Name	Movie	Duration
1	Hosanna	VTV	300
2	Kanimaa	Retro	400
3	Sawadeeka	Vidaamuyarchi	250
4	Mundhinam	Vaaranam Aayiram	300
5	Kannai Vittu	Iru Mugan	400
6	Jinguchaa	Thug Life	350
7	Golden Sparrow	NEEK	350
8	Vaseegara	Minnale	400

Trying to list songs with invalid user

Name	Description
uname * required string (query)	Rajesh
<button>Execute</button>	

Code	Details
401 <i>Undocumented</i>	Error: Unauthorized Response body <pre>{ "detail": "Invalid username or password" }</pre> Response headers <pre>content-length: 41 content-type: application/json date: Tue, 19 Aug 2025 12:17:13 GMT server: uvicorn</pre>

Retrieve specific song with valid user

Name	Description
_id * required integer (path)	5
uname * required string (query)	Nandhini

Execute

Code	Details
200	Response body <pre>{ "Duration": 400, "Name": "Kannai Vittu", "Id": 5, "Movie": "Iru Mugan" }</pre> Response headers <pre>content-length: 65 content-type: application/json date: Tue, 19 Aug 2025 12:19:32 GMT server: uvicorn</pre>

Retrieve specific song with valid user but with invalid song id

Name	Description
_id * required integer (path)	10
uname * required string (query)	Nandhini

Execute

404
Undocumented

Error: Not Found

Response body

```
{  
  "detail": "Song not found"  
}
```

Response headers

```
content-length: 27  
content-type: application/json  
date: Tue, 19 Aug 2025 12:20:08 GMT  
server: uvicorn
```

Trying to retrieve specific song with invalid user

Name	Description
_id * required integer (path)	5
uname * required string (query)	Rajesh

[Execute](#)

Code	Details
401 <i>Undocumented</i>	Error: Unauthorized Response body <pre>{ "detail": "Invalid username or password" }</pre> Response headers <pre>content-length: 41 content-type: application/json date: Tue, 19 Aug 2025 12:20:47 GMT server: uvicorn</pre>